

BTS CIEL

Cybersécurité, Informatique et réseaux, Électronique

Option A : « Informatique et réseaux »

Épreuve professionnelle E6

Valorisation de la donnée et cybersécurité

Dossier de réalisation du projet

Groupement académique : Besançon-Dijon	Session : 2025
Lycée ou Centre de formation : Lycée Jules Haag	
Ville : BESANÇON	
Nom du projet :	Serre Automatisée
Projet industriel :	OUI NON

Equipe de développement.

Professeur :	L. LE JONCOUR – G. FRAPSAUCE – F. GUILLET – O. HACQUARD
L'entreprise :	Lycée Jules Haag
Etudiant 1 :	Tep Raphael
Etudiant 2 :	Chebira Mous'ab

Enlever les lignes inutiles.



Sommaire

1	CAHIER DES CHARGES.....	3
2	ANALYSE ET SPÉCIFICATIONS.....	3
2.1	LES CAS D'UTILISATION AVEC LEURS SCÉNARIOS MODIFIÉS PENDANT L'ÉTUDE.....	3
2.2	LE DIAGRAMME D'EXIGENCE DU SYSTÈME.....	3
2.3	TESTS DE VALIDATIONS.....	3
3	DÉFINITION DES ÉCHANGES ENTRE LES SOUS-SYSTÈMES.....	3
4	PRÉSENTATION DU TRAVAIL DU CANDIDAT N°1.....	4
4.1	DÉCOUVERTE(S) MATÉRIELLE(S).....	4
4.2	DÉCOUVERTE(S) LOGICIELLE(S).....	4
4.3	CHOIX DE MATÉRIEL.....	4
4.4	PROTOTYPAGE ET MAQUETTAGE.....	4
4.5	CONCEPTION PRÉLIMINAIRE.....	4
4.5.1	<i>Liste des objets.....</i>	<i>4</i>
4.5.2	<i>Diagrammes de séquences entre objet.....</i>	<i>4</i>
4.6	CONCEPTION DÉTAILLÉE.....	4
4.6.1	<i>Algorithme de la méthode</i>	<i>5</i>
4.6.2	<i>Test unitaire de l'algorithme de la méthode</i>	<i>5</i>
4.6.3	<i>Algorithme de la méthode</i>	<i>5</i>
4.6.4	<i>Test unitaire de l'algorithme de la méthode</i>	<i>5</i>
4.6.5	<i>Etc.....</i>	<i>5</i>
4.7	CODAGE.....	5
4.8	RÉSULTATS DES TESTS UNITAIRES.....	5
4.9	BILAN PERSONNEL.....	5
4.9.1	<i>Planning réalisé.....</i>	<i>5</i>
4.9.2	<i>Difficultés rencontrées.....</i>	<i>5</i>
5	PRÉSENTATION DU TRAVAIL DU CANDIDAT N°.....	6
6	INTÉGRATION ET TESTS.....	6
6.1	INTÉGRATION.....	6
6.2	TESTS.....	6
7	VALIDATION DES ATTENTES DU POINT DE VUE CLIENT.....	6
8	BILAN TECHNIQUE.....	6
8.1	CRITIQUE(S) DU PROJET.....	6
8.2	ÉVOLUTION(S) POSSIBLE(S).....	6
8.3	ÉCHANGES AVEC LES INDUSTRIELS ET/OU LES VENDEURS.....	6
9	DOSSIER NUMÉRIQUE.....	6

1 Cahier des charges

Pour le projet de gestion de Serre automatique dont le but est de faire pousser des plantes en contrôlant leurs paramètres environnementaux et en les régulant. Leur gestion inclut : le maintien de l'atmosphère à un taux d'humidité suffisant, que la terre est assez humidifiée pour que la plante en son sein pousse correctement et enfin il faut réguler la température pour qu'elle soit constante.

Le système doit utiliser de l'eau de pluie déminéralisée et il faut pouvoir observer l'évolution du taux d'humidité dans l'air, dans la terre et de la température.

On utilisera un Raspberry pi connecté à un ESP32 dans une maquette de mini serre montée.

Dans la serre il y aura : 2 pompes, 2 capteurs d'humidités, un volet mobile déjà avec la serre, de même pour le plancher chauffant. Il y aura aussi un brumisateur, un ventilateur et un capteur de température/hygrométrie.

Pour la connexion au Raspberry il faut que le mot de passe soit d'au moins 12 caractères alphanumériques

Travail d'équipe

Doit être reformulé avec vos mots à vous

Pas de copier-coller

Faire éventuellement un résumé du cahier des charges :

- La problématique.
- L'expression des besoins.
- La structure matérielle du système.

Présenter le rôle de chaque membre de l'équipe pour réaliser le projet en fonction de ce cahier des charges.

Automatisation d'une serre connectée

2 Analyse et spécifications

Travail d'équipe

2.1 Les cas d'utilisation avec leurs scénarios modifiés pendant l'étude

Présenter les modifications qui ont été apportées tout au long du projet.

Les modifications devront être identifiées.

2.2 Le diagramme d'exigence du système

Présenter les modifications et les ajouts qui ont été apportées tout au long du projet. Les modifications devront être identifiées.

2.3 Tests de validations

Les tests de validation découlent des différents scénarios à respecter.

Les tests de validation peuvent être présentés sous la forme de tableaux :

Action à tester	Tests à réaliser	Résultat(s) attendu(s)
Quelle fonctionnalité du système je souhaite tester, à partir de « Atteinte des objectifs du point de vue client » ? (voir dossier de présentation)	Comment je vais réaliser le test <u>concrètement et précisément</u> ? Comment je m'y prends ?	Qu'est-ce que je dois observer <u>concrètement</u> pour dire que ça marche ?

Etc.

Dans ce tableau, mettre 1 ligne par item de la recette du point de vue client.

Ces tests permettent de montrer au client que le système que vous lui livrez fonctionne et répond à sa demande et à ses besoins.

Les colonnes 2 et 3 doivent être bien différenciées.

3 Définition des échanges entre les sous-systèmes

Les échanges entre sous-systèmes sont par exemple les formats de fichier, formats de trame, etc... qui peuvent être précisés par des diagrammes conçus en communs (diagramme d'activité, diagramme de déploiement, interface, etc.).

Présenter les diagrammes réalisés obligatoirement avec un AGL.

Travail d'équipe

Décrire par exemple :

- requête HTTP envoyée par un client à un serveur, et réponse du serveur
- trames échangées entre un client et un serveur TCP
- trames UDP envoyées en diffusion
- trames MODBUS échangées entre maître et esclaves

Détailler précisément le format des trames (chaines de caractères).

1) Échanges entre les sous-systèmes

Dans le cadre de ce projet, plusieurs sous-systèmes communiquent entre eux afin d'assurer l'acquisition, le stockage et l'affichage des données environnementales de la serre. Ces échanges sont réalisés par des protocoles simples (HTTP et SQL) et automatisés grâce à un script PHP programmé en tâche planifiée via CRON sur le Raspberry Pi.

Les sous-systèmes identifiés sont :

- ESP32 : microcontrôleur qui effectue les mesures via les capteurs (température, hygrométrie, humidité des plantes) et contrôle les différents actionneurs de la serre (pompe, brumisateur, ventilateur, plancher chauffant volet ouvrant).
- Raspberry Pi : serveur local qui interroge l'ESP32, traite les données et les stocke.
- Base de données MySQL : stockage des mesures, des seuils, et des identifiants.
- Interface web (PHP/JS/HTML) : permet d'afficher les données, les exporter, ou modifier les seuils (si connecté).

2) Exemple d'échange

Toutes les 5 minutes, une tâche CRON exécute le fichier *acquerirDonnees.php*.

Celui-ci :

- envoie une requête HTTP GET à l'ESP32
- reçoit une chaîne de type 23.1;59.2;60.0;45.8 (correspondant à : température ; humidité plante 1 ; humidité plante 2 ; hygrométrie)
- décode cette trame
- insère les données dans la base de données via une requête SQL

3) Détails des échanges

Source	Destination	Type d'échange	Protocole	Format
Raspberry Pi	ESP32	Requête de données	HTTP GET	/mesures→réponse: 23,4;61,0;58,2;43,0
Raspberry Pi	Base MySQL	Insertion	SQL	INSERT INTO Mesures (,,) VALUES (,,)
Interface web	Base MySQL	Lecture pour affichage	SQL	SELECT * FROM Mesures → JSON pour Chart.js
Interface web	Base MySQL	Modification des seuils	SQL	UPDATE Seuils SET ...
Interface web	Base MySQL	Export CSV	SQL	SELECT * FROM Mesures →

4) Format de la trame ESP32

Trame reçue (texte brut) : 24.1;65.2;62.9;46.5

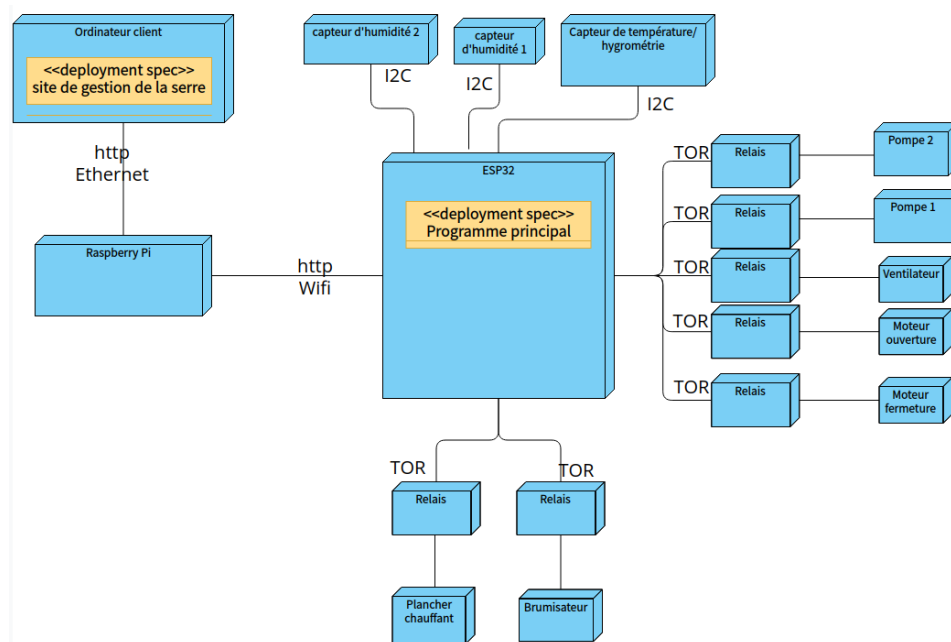
Ordre :

- 1) Température (°C)
- 2) Humidité plante 1 (%)
- 3) Humidité plante 2 (%)
- 4) Hygrométrie (%)

Les données sont séparées par un point-virgule (;) et analysées avec la fonction explode() en PHP.

5) Diagramme de déploiement

Le diagramme suivant illustre la disposition physique et logique des sous-systèmes du projet, ainsi que les types de liaisons entre eux :



L'ESP32 est connecté aux capteurs et actionneurs par des liaisons série.

Il agit comme point d'accès Wi-Fi, et le Raspberry Pi s'y connecte pour récupérer les données. Le Raspberry héberge également l'interface web et communique avec un navigateur client via HTTP.

L'ensemble est localisé sur le réseau local, sans besoin de connexion Internet.

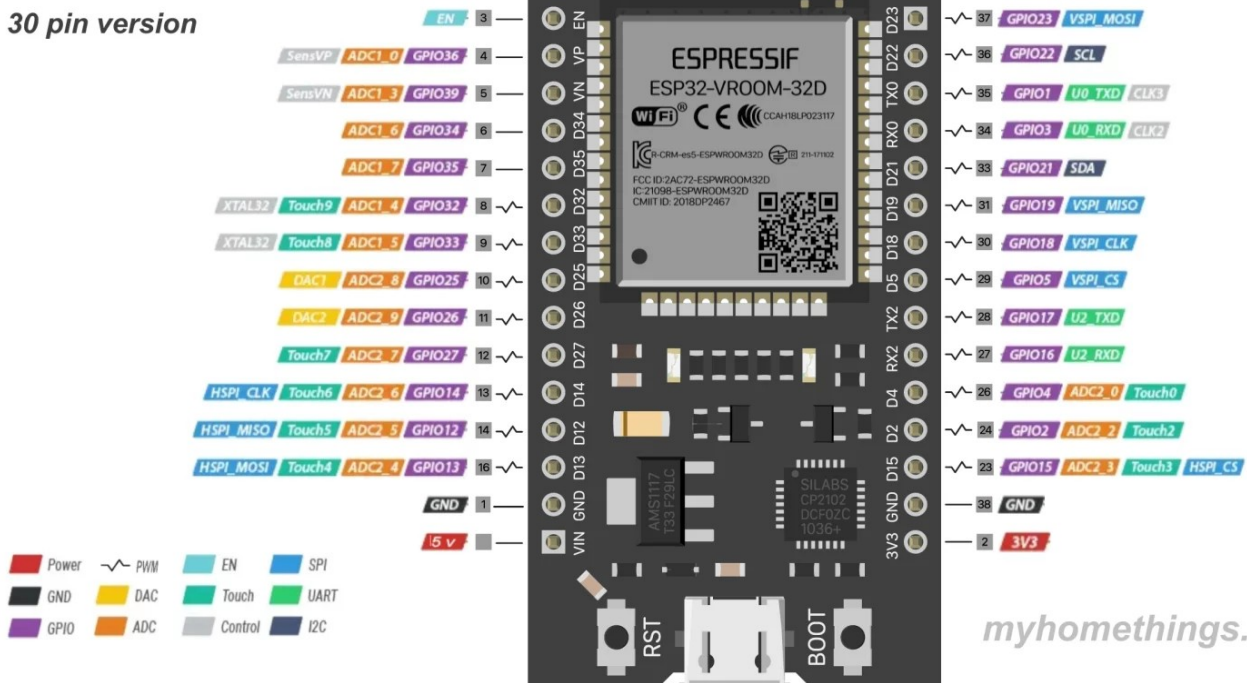
4 Présentation du travail du candidat N°1

4.1 Découverte(s) matérielle(s)

4.1.1. L'ESP32 VROOM

ESP-WROOM-32 Development Board

30 pin version



les différentes étapes de l'apprentissage et de la compréhension du fonctionnement de l'ESP32 :

- Apprendre à cabler un ESP et à me servir d'une breadboard. Je me suis très peu servi d'arduinios auparavant, j'ai du apprendre tout le fonctionnement d'un tel système et j'ai passé beaucoup de temps à comprendre et à prendre en main ce système.

4.1.2. Différents types de capteurs et actionneurs :

J'ai du également apprendre à me servir de différents capteurs et actionneurs. Le seul capteur que j'avais utilisé par le passé était un capteur d'humidité et de température en DHT11. La liste des capteurs et actionneurs que j'ai appris à utiliser sont :

- Les capteurs d'humidité au sol
- Les microrupteurs
- Le ventilateur
- Les pompes d'arrosage
- Les brumisateurs
- Les moteurs
- Le plancher chauffant
- Les différents relais

Voir documentation de la serre pour plus d'information

<https://www.a4telechargement.fr/BE-SER/D-BE-SER-Technique-v2020.pdf>

4.2 Choix de matériel

Le matériel utilisé est une maquette de mini-serre automatisée ainsi qu'un ESP 32.

L'ESP32 permet de brancher de nombreux cables a ses différentes broches, permettant ainsi un grand nombre de connexions.

Au total, l'ESP32 est connecté a 5 capteurs (2 capteurs d'humidité au sol, 1 capteur d'humidité température et 2 microrupteurs)

il est également connecté a 7 actionneurs (2 pompes, 1 brumisateur, 1 ventilateur, a deux moteurs et au plancher chauffant.

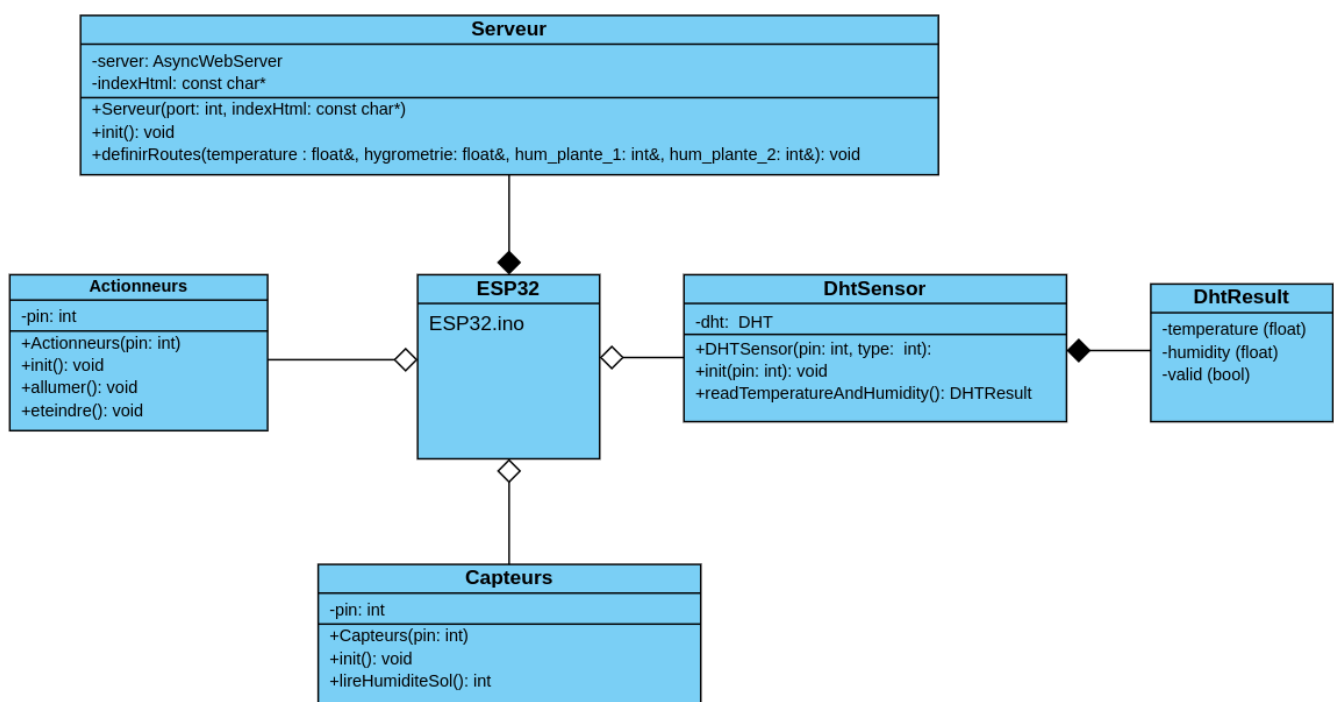
L'ESP32 est également peu onéreux et peut donc être remplacé facilement en cas de problème (voir partie 4.8.2 sur les difficultés rencontrées pour plus de détails.). sur amazon, il est disponible a 9€ unoté et 26€ le lot de 5 (5€20 par ESP32) contrairement a ce qui est recommandé sur la documentation de la serre : L'AutoProg X2 K-APV2-M vendu par les créateurs de la serre qui coute 130€ non monté et 210€ monté.

Le prix d'un seul exemplaire permettrait d'acheter plus de 25ESP32.

Produit	Prix	Avantages	Inconvénient
ESP32	8€	Petit, peu couteux, grand nombre d'emplacements GPIO, puissance largement suffisante	Besoin d'une breadbox pour brancher les câbles
Boitier Arduino	25€	Prix raisonnable, facile d'utilisation, grande documentation	Pas assez d'emplacements GPIO et pas assez puissant
AutoProg X2	130€ / 210€	Utilisation simple avec la serre, grande documentation donnée sur le site de la serre, installation facile	Très cher, a monter soi-même si on souhaite payer moins cher

4.3 Conception préliminaire

Diagramme de classe



4.3.1 Liste des objets

Objet / blocs	Rôle(s)	Issu de la classe
serveur	Crée un serveur accessible en Wifi sur l'ESP32 contenant les données acquises (temperature, humidité des 2 plantes et hygrométrie)	Serveur
terre_1	Renvoie le taux d'humidité au sol de la plante numéro 1	Capteurs
terre_2	Renvoie le taux d'humidité au sol de la plante numéro 1	Capteurs
dhtsensor	Renvoie le taux d'hygrométrie et la température de la serre	DhtSensor
pompe	Envoie un signal au relais pour lui dire de laisser passer le courant vers la pompe	Actionneurs
brumisateur	Envoie un signal au relais pour lui dire de laisser passer le courant vers le brumisateur	Actionneurs
ventilateur	Envoie un signal au relais pour lui dire de laisser passer le courant vers le ventilateur	Actionneurs
moteur_open	Envoie un signal au relais pour lui dire de laisser passer le courant vers le moteur qui ouvre la fenetre de la serre	Actionneurs
moteur_close	Envoie un signal au relais pour lui dire de laisser passer le courant vers le moteur qui ferme la fenetre de la serre	Actionneurs
plancher	Envoie un signal au relais pour lui dire de laisser passer le courant vers le plancher chauffant	Actionneurs

4.3.2 Diagrammes de séquences entre objet

Diagramme de séquence de la boucle principale

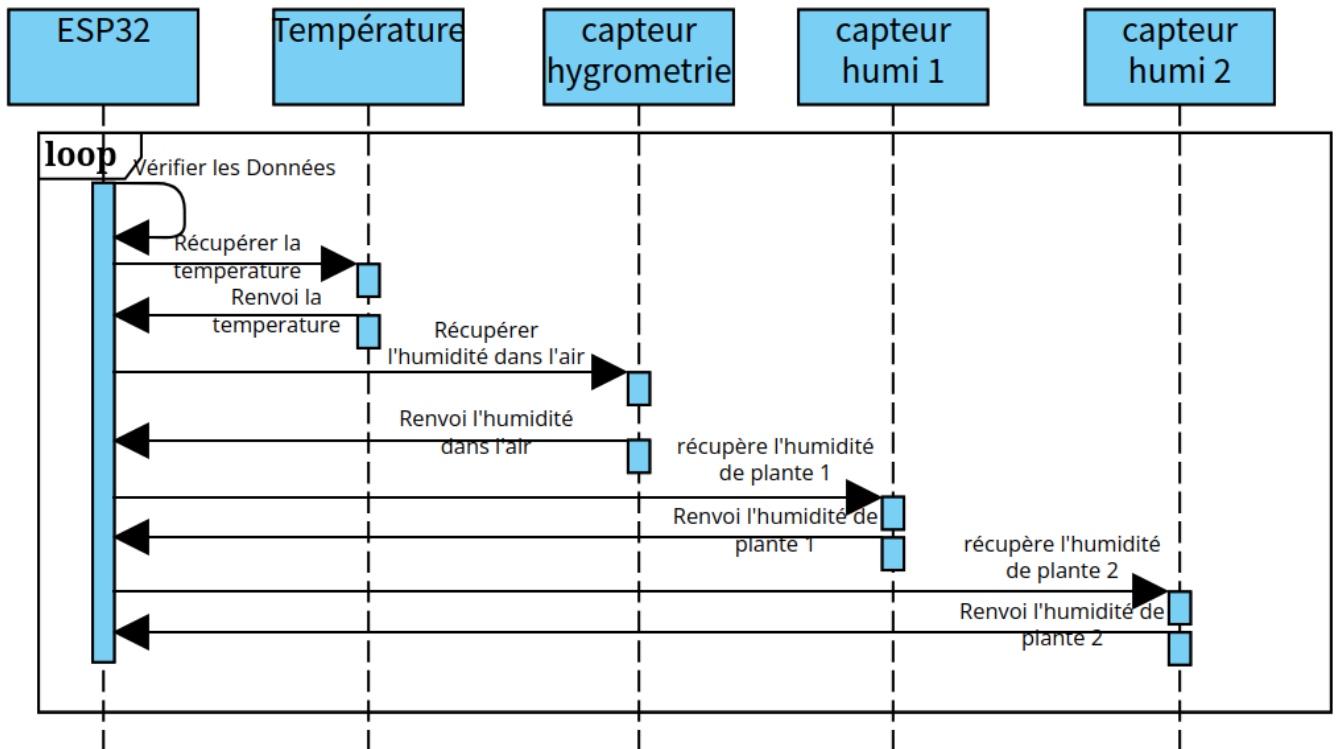


Diagramme de séquence cas particuliers : température trop élevée

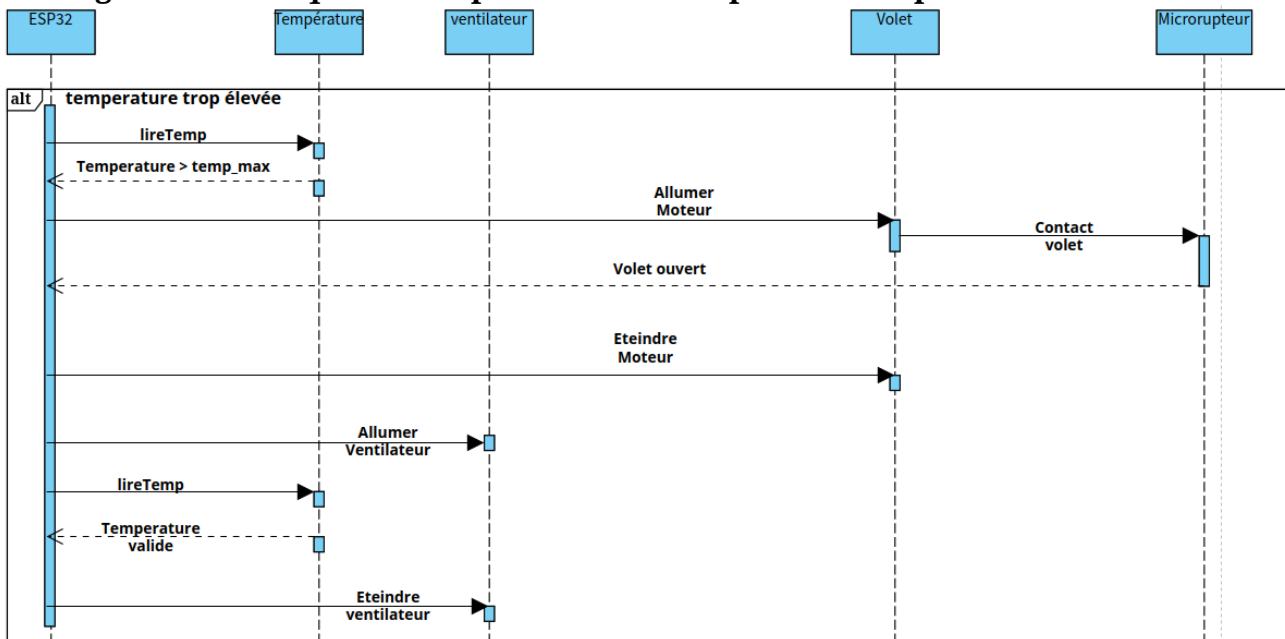


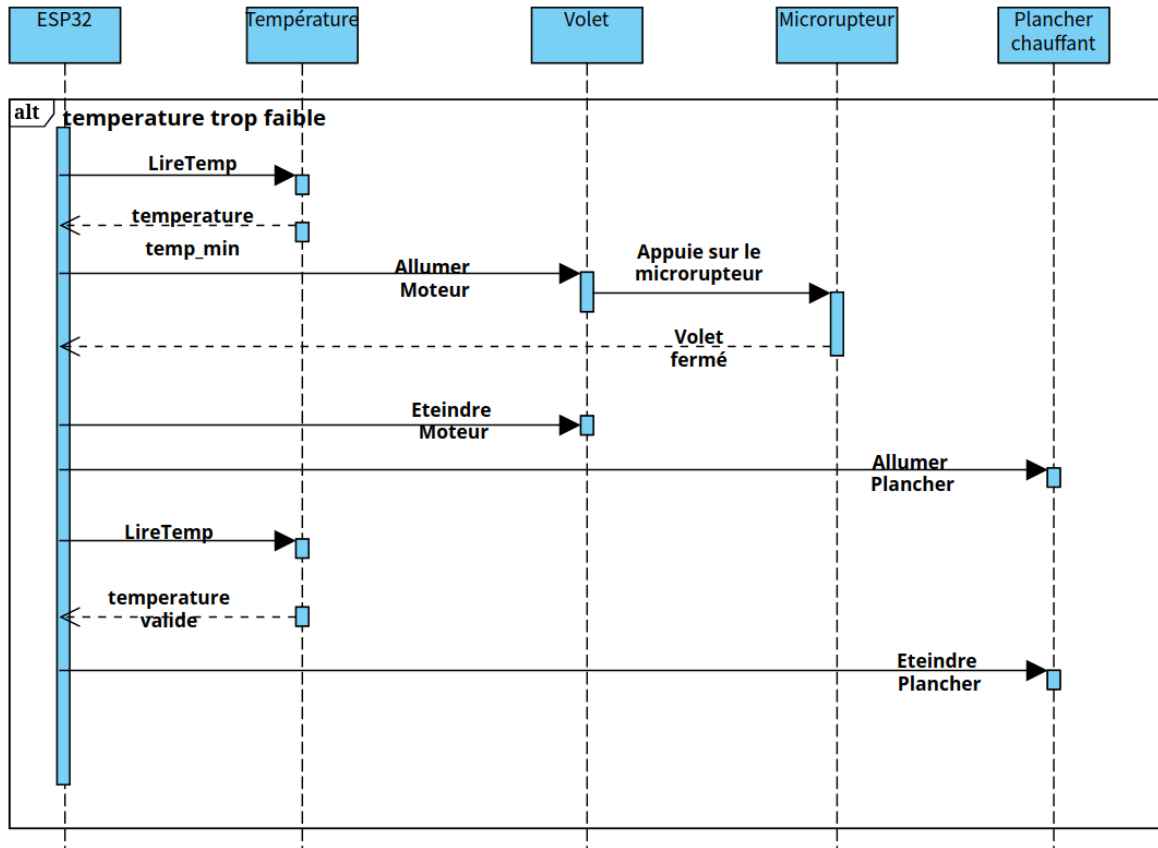
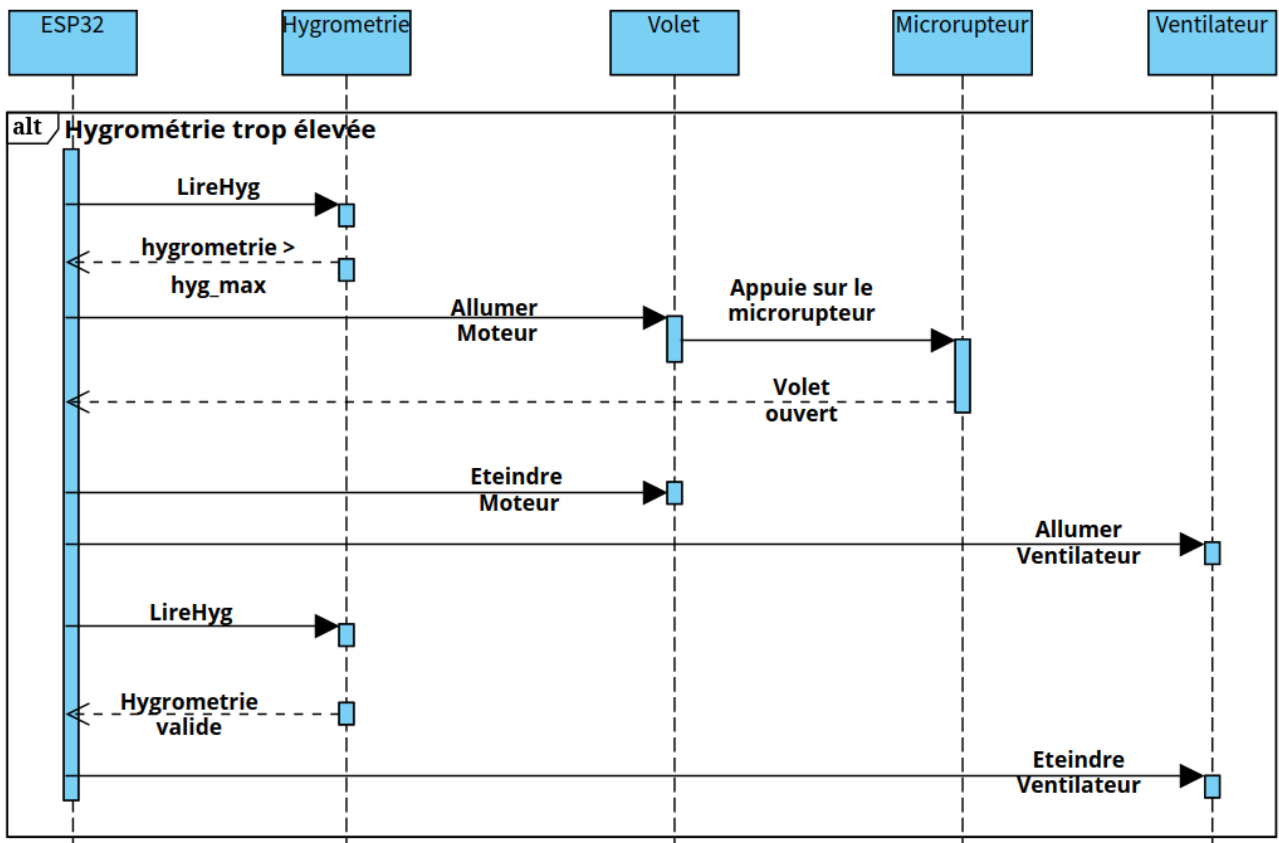
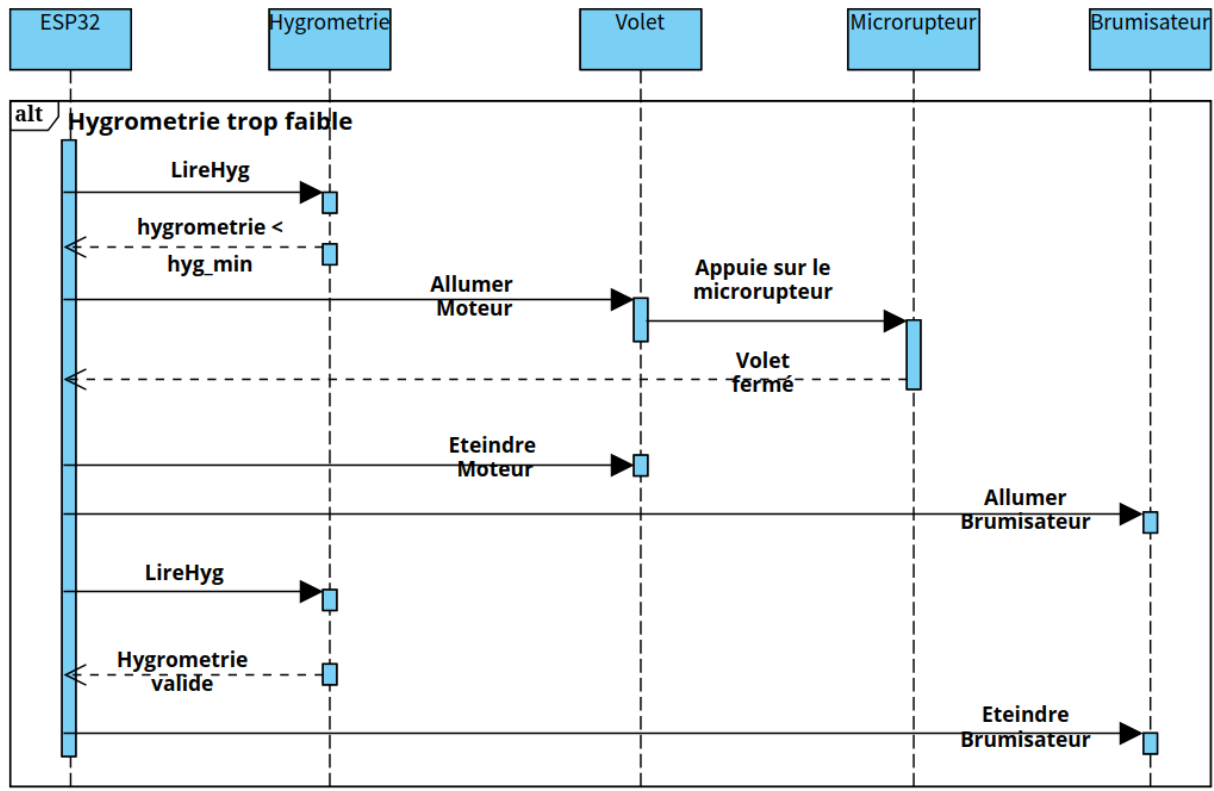
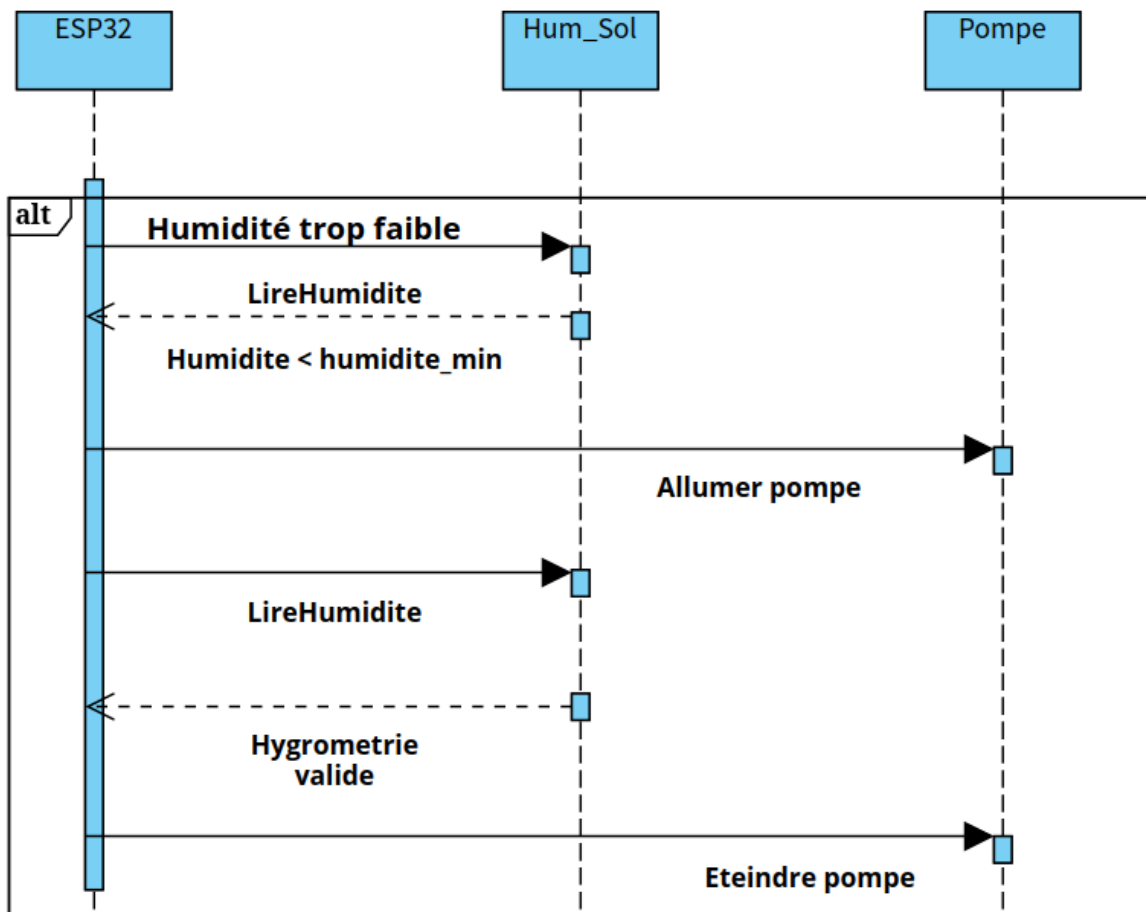
Diagramme de séquence cas particuliers : température trop faible**Diagramme de séquence cas particuliers : hygrométrie trop élevée**

Diagramme de séquence cas particuliers : Hygrométrie trop faible**Diagramme de séquence cas particuliers : humidité au sol trop faible**

4.4 Conception détaillée

4.4.1 Liste des tests unitaires

Liste des tests pour les différents capteurs

Action à tester	Tests à réaliser	Résultat(s) attendu(s)
Lecture hygrométrie air	Exposer le capteur à un environnement sec, puis humide	Valeur faible dans le sec (< 40%) et élevée dans l'humide (> 60%)
Lecture température air	Placer le capteur dans un environnement chaud puis froid	Valeur augmente en chaleur, diminue au froid (ex: 25°C puis 10°C)
Lecture humidité du sol	Tester capteur dans terre sèche, légèrement humide, et saturée d'eau	Lecture faible, moyenne, puis élevée selon l'humidité du sol

Liste des tests pour les différents actionneurs

Action à tester	Tests à réaliser	Résultat(s) attendu(s)
Activation pompe	Simuler une demande d'arrosage ou activer manuellement	Pompe s'allume, de l'eau coule jusqu'aux plantes
Désactivation pompe	Arrêter manuellement ou via seuil d'humidité suffisant	Pompe s'éteint, arrêt de l'écoulement
Activation brumisateur	Simuler air trop sec ou activer manuellement	Le brumisateur s'allume, émission de brume visible
Désactivation brumisateur	Simuler hygrométrie atteinte (> seuil) ou arrêt manuel	Brumisateur s'éteint, plus d'humidification
Activation ventilateur	Simuler température élevée ou activer manuellement	Ventilateur souffle de l'air
Désactivation ventilateur	Simuler retour à température normale	Ventilateur s'arrête
Activation plancher chauffant	Simuler température basse ou activer manuellement	Chauffage s'active, plancher devient plus chaud (testable avec capteur)
Désactivation plancher chauffant	Simuler température adéquate ou arrêt manuel	Chauffage s'éteint, plus de montée de température
Activation moteur	Activer manuellement ou via programme	Moteur tourne dans le bon sens à la vitesse attendue
Réversibilité moteur	Commander changement de sens	Moteur change de direction
Arrêt moteur	Arrêter manuellement ou automatiquement	Moteur s'arrête

4.4.2 Liste des tests fonctionnels

Action à tester	Test à réaliser	Résultat attendu
Activer le plancher chauffant s'il fait froid	Simuler une température < 15°C avec le capteur ; vérifier que le signal de chauffage est bien déclenché	Le plancher chauffant s'allume automatiquement
Activer le ventilateur s'il fait chaud	Simuler une température > 28°C ; vérifier que le ventilateur reçoit un signal d'activation	Le ventilateur se met en marche pour rafraîchir l'air
Activer le brumisateuse s'il fait trop sec	Simuler une hygrométrie < 40 % ; observer l'activation du brumisateuse via la classe d'actionneur	Le brumisateuse se déclenche pour humidifier l'air
Activer le ventilateur et les moteurs du volet s'il fait trop humide	Simuler une hygrométrie > 80 % ; vérifier que les deux actionneurs (ventilateur + moteurs du volet) reçoivent un ordre d'activation	Le ventilateur souffle et les volets s'ouvrent pour évacuer l'humidité

4.5 Codage

Tous les fichiers de code de l'ESP32, classes incluses se trouvent dans ce dépôt github :

https://github.com/RaphaelTep/Serre_Archive/tree/ESP32

Instruction de test du code :

Pour utiliser le code il faut lancer l'IDE Arduino, aller dans « file » puis « preference » et entrer ce lien dans « Additional boards manager URLs :

https://dl.espressif.com/dl/package_esp32_index.json

Une fois entré, il faut aller dans le Boards manager, tapez « esp32 » et téléchargez le board manager « esp32 » par Espressif Systems

il faut ensuite télécharger les différentes librairies dans le Library Manager :

ALPkc_opta par Arduino

Arduino RS485 par Arduino

Arduino_DebugUtils par Arduino

Arduino_Opta_Blueprint par Daniele

Arduino_SerialUpdater par Arduino

Adafruit Unified Sensor par Adafruit

ArduinoJson par Benoit Blanchon

Async TCP par ESP32Async

AsyncEspFsWebserver par Tolentino

DHT sensor library par Adafruit

ESPAsyncWebServer par Lacamera

Une fois toutes ces librairies installées, il ne reste plus qu'à brancher l'ESP, vérifier les branchements et le programme est téléversable depuis l'IDE Arduino.

4.6 Résultats des tests unitaires

Action à tester	Test à réaliser	Résultat attendu	Résultat obtenu	Remèdes
Activation pompe	Simuler une demande d'arrosage ou activer manuellement	Pompe s'allume, de l'eau coule jusqu'aux plantes	Pompe activée automatiquement après détection d'humidité basse, écoulement observé	
Désactivation pompe	Arrêter manuellement ou via seuil d'humidité suffisant	Pompe s'éteint, arrêt de l'écoulement	Pompe arrêtée automatiquement dès que l'humidité atteint le seuil, plus d'écoulement	
Activation brumisateur	Simuler air trop sec ou activer manuellement	Le brumisateur s'allume, émission de brume visible	Brumisateur activé à < 45% humidité, brume visible immédiatement	
Désactivation brumisateur	Simuler hygrométrie atteinte (> seuil) ou arrêt manuel	Brumisateur s'éteint, plus d'humidification	Brumisateur arrêté automatiquement à 60% humidité, plus de vapeur visible	
Activation ventilateur	Simuler température élevée ou activer manuellement	Ventilateur souffle de l'air	Ventilateur activé à > 28°C, flux d'air détecté	
Désactivation ventilateur	Simuler retour à température normale	Ventilateur s'arrête	Ventilateur désactivé automatiquement à 24°C	
Activation plancher chauffant	Simuler température basse ou activer manuellement	Chauffage s'active, plancher devient plus chaud (testable avec capteur)	Chauffage actuellement non utilisable	Déplacer/ajouter un relais plus proche pour brancher le plancher chauffant
Désactivation plancher chauffant	Simuler température adéquate ou arrêt manuel	Chauffage s'éteint, plus de montée de température	Chauffage actuellement non utilisable	Déplacer/ajouter un relais plus proche pour brancher le plancher chauffant
Activation moteur	Activer manuellement ou via programme	Moteur tourne dans le bon sens à la vitesse attendue	Moteur tourne à dans le sens prévu	
Réversibilité moteur	Commander changement de sens	Moteur change de direction	Inversion confirmée, rotation en sens opposé observée	
Arrêt moteur	Arrêter manuellement ou automatiquement	Moteur s'arrête	Rotation stoppée sans à-coups, arrêt complet confirmé	

4.7 Bilan personnel

4.7.1 Planning réalisé

Etat actuel du projet :

Le projet est fini en très grande partie, il ne reste que quelques modifications sur les actionneurs (régler un bug avec les moteurs) et modifier la serre pour pouvoir brancher le plancher chauffant qui devrait être fonctionnel car il suffit d'alimenter le relais et de le couper une fois la température suffisante.

Les capteurs sont entièrement fonctionnels, il manque juste un étalonnage au niveau du capteur d'humidité au sol pour le convertir en pourcentage.

Le capteur d'hygrométrie et de température ont en plus un système qui empêche d'utiliser des valeurs erronées. Les informations sont bien transmises par un JSON sur le site de l'ESP32.

Le travail et l'avancée est notée du début du projet jusqu'au 1^{er} avril sur un document libre office writer. La suite n'étant que des modifications (passage du code en classe) ou des changements sans grand avancement sur le projet en général, je ne les ai pas noté. J'ai profité de cette période pour travailler sur la présentation finale et faire de la documentation.

4.7.2 Difficultés rencontrées

3 types de difficultés ont été rencontrées lors de ce projet :

Premièrement, c'était la première fois que j'utilisais ce genre de systèmes, je n'avais jamais touché à un ESP32, très rarement essayé un Arduino également, je n'étais donc pas familier avec cet environnement de travail. J'ai par la suite commencé à prendre mes marques et utilisant par exemple beaucoup plus de documentations et ai pris le réflexe d'aller voir directement la documentation du matériel utilisé.

Ensuite, j'ai eu des problèmes de matériels, et en grande quantité :

- L'alimentation de l'ESP32 : sur le premier ESP32 qui était originellement attaché à la serre, le câble tenait mal et le port de charge était en mauvaise état. Après quelques temps, il s'est détaché et impossible de le remettre en état de marche. Le changement d'ESP32 a également débranché certains câbles car le support pour la breadbox n'est pas stable (chose dont je n'étais pas au courant) au préalable.

- La surchauffe de l'ESP32 : Le second ESP32 est celui qui m'a permis d'avancer le plus, en effet, le code que j'avais à ce moment là est celui qui était quasiment fonctionnel avant le passage en classe et l'écriture dans l'EEPROM. Lors d'un test de l'un des actionneurs, il m'a été recommandé d'essayer de ne pas passer par le relais et directement par la plaque d'alimentation. Lors du test, l'ESP32 a surchauffé et a refusé de fonctionner pendant plus d'1min30 sans surchauffer à nouveau, le rendant donc inutilisable pour le projet.

- Problèmes de câbles : en dehors des câbles que je ne peux pas brancher pour le plancher chauffant, le câble d'alimentation du ventilateur de la serre s'est retrouvé coincé dans son relais et son embout s'est arraché, me forçant à utiliser des stratégies inhabituelles (fusionner le bout encore valide du câble et le mettre dans l'embout d'un autre câble en bon état) pour le faire fonctionner.

Enfin, en termes de développement pur, j'ai pris un temps absolument monstrueux pour faire des classes fonctionnelles par rapport au code que j'avais de base avec notamment une classe DhtSensor qui marchait parfaitement sur un autre système mais pas dans la Serre.

5 Présentation du travail du candidat N°2

5.1 Découvertes matérielles

Pour réaliser ce projet, j'ai été amené à manipuler le matériel indispensable à la mise en œuvre d'un serveur embarqué. J'ai principalement utilisé un Raspberry Pi 3B+, un mini-ordinateur doté de 1 Go de RAM, de plusieurs ports USB, d'un port HDMI et d'un port Ethernet.

Lors de la configuration initiale, j'ai connecté le Raspberry à un écran via un câble HDMI, et utilisé un clavier et une souris USB pour accéder à l'environnement graphique de Raspbian. L'alimentation a été assurée par un chargeur USB-C 5V 3A, garantissant une alimentation stable.

J'ai installé le système d'exploitation Raspbian OS sur une carte micro-SD de 32 Go, à l'aide du logiciel Balena Etcher. L'ensemble des fichiers de configuration, le site web, la base de données ainsi que les scripts PHP ont été stockés sur cette carte.

Le Raspberry Pi s'est ensuite connecté au réseau Wi-Fi local généré par l'ESP32, configuré en point d'accès. Cela m'a permis de travailler en autonomie complète, sans besoin de connexion Internet.

5.2 Découvertes logicielles

Sur le plan logiciel, j'ai transformé le Raspberry Pi en serveur complet et autonome. Les étapes ont été les suivantes :

1) Installation des services

Dans le terminal, j'ai utilisé les commandes suivantes :

```
sudo apt update  
sudo apt install apache2 php libapache2-mod-php mysql-server php-mysql phpmyadmin
```

J'ai configuré phpMyAdmin pour gérer la base plus facilement et vérifié que Apache fonctionnait à l'aide de :

```
sudo systemctl status apache2
```

2) Automatisation avec CRON

J'ai appris à planifier des tâches avec `crontab -e` et à exécuter mon script PHP toutes les 5 minutes :

```
* /5 * * * * /usr/bin/php /var/www/html/acquerirDonnees.php
```

3) Requêtes HTTP avec `wget`

J'ai utilisé cette commande pour interroger manuellement l'ESP32 :

```
wget -qO- http://192.168.4.1
```

Elle m'a permis de visualiser directement la trame reçue :

```
24.5;59.8;60.2;41.0
```

4) Sécurité et sessions

J'ai mis en place :

- la gestion de sessions PHP (`session_start()`)
- le hachage des mots de passe (`password_hash()`)
- la vérification sécurisée (`password_verify()`)
- la redirection automatique vers `login.php` si l'utilisateur n'est pas authentifié.

5) Affichage graphique

Pour afficher les mesures sur le site, j'ai utilisé `Chart.js` avec des objets JSON générés par PHP. Cela m'a permis de suivre l'évolution en temps réel des conditions dans la serre (température, humidité plante 1, humidité plante 2, hygrométrie).

5.3 Choix de matériel

Le Raspberry Pi a été choisi pour sa capacité à répondre aux besoins suivants :

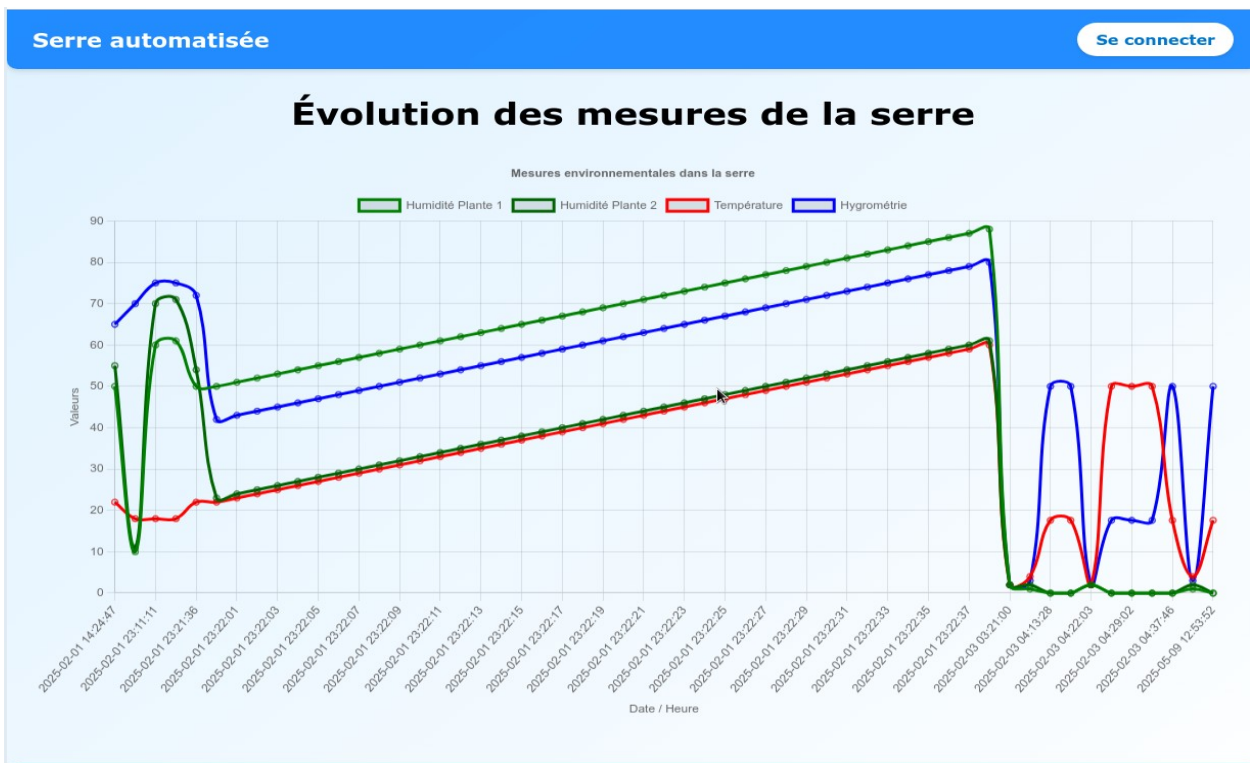
- Exécution d'un système Linux embarqué (Raspbian)
- Hébergement d'un site web local accessible via navigateur
- Connexion autonome à un réseau local (créé par l'ESP32)
- Coût réduit, faible encombrement, et bonne fiabilité.

Il représente une solution idéale pour un déploiement pédagogique.

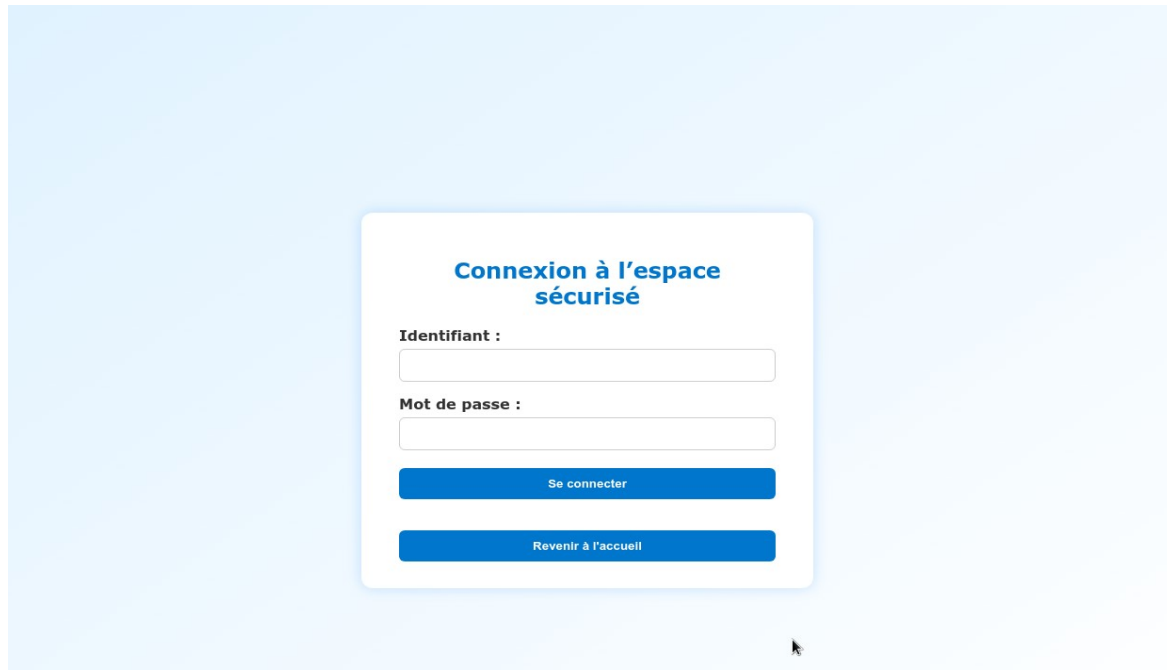
5.4 Prototypage et maquettage

J'ai développé une interface web locale complète, constituée des pages suivantes. Les captures suivantes illustrent les principales pages de l'interface web que j'ai développée :

- `index.php` : affichage graphique des données via Chart.js



- `login.php` : page d'identification sécurisée



Connexion à l'espace sécurisé

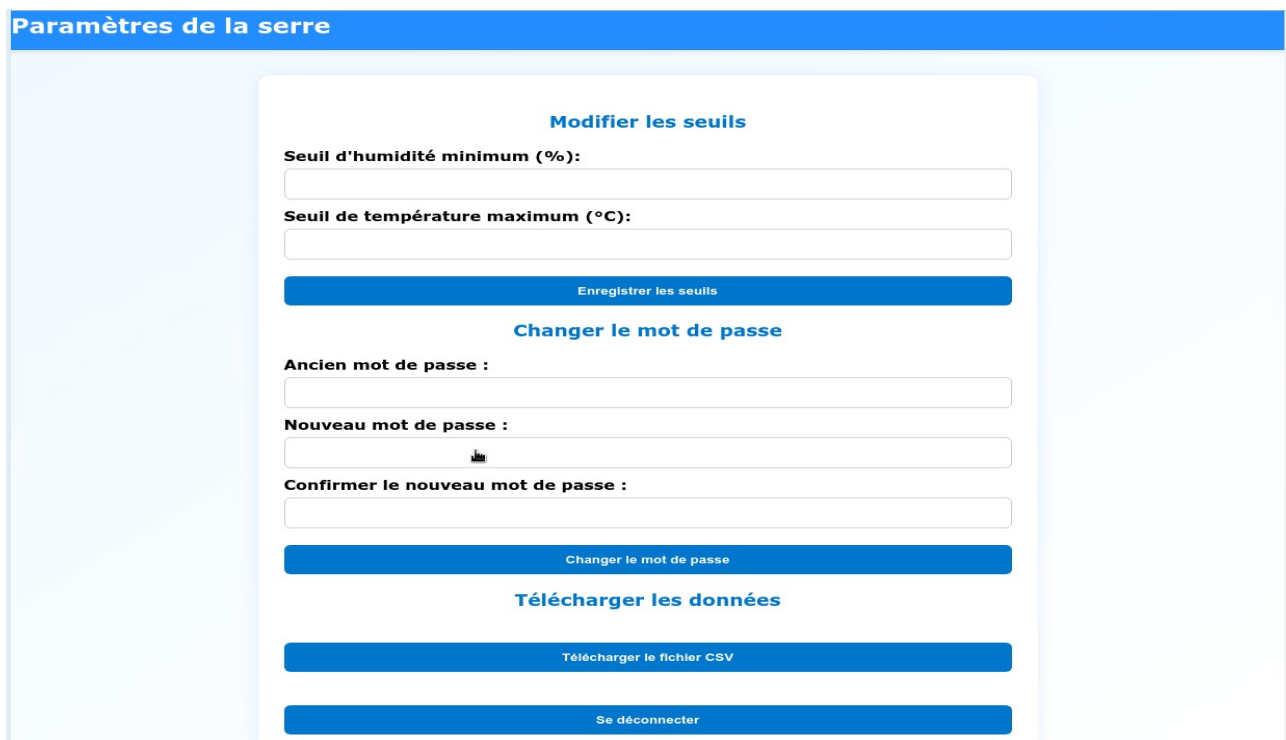
Identifiant :

Mot de passe :

Se connecter

Revenir à l'accueil

- `parametres.php` : page accessible uniquement après connexion, contenant :
 - le formulaire de modification des seuils,
 - le bouton de téléchargement des mesures au format CSV,
 - le formulaire de changement de mot de passe.



Paramètres de la serre

Modifier les seuils

Seuil d'humidité minimum (%):

Seuil de température maximum (°C):

Enregistrer les seuils

Changer le mot de passe

Ancien mot de passe :

Nouveau mot de passe :

Confirmer le nouveau mot de passe :

Changer le mot de passe

Télécharger les données

Télécharger le fichier CSV

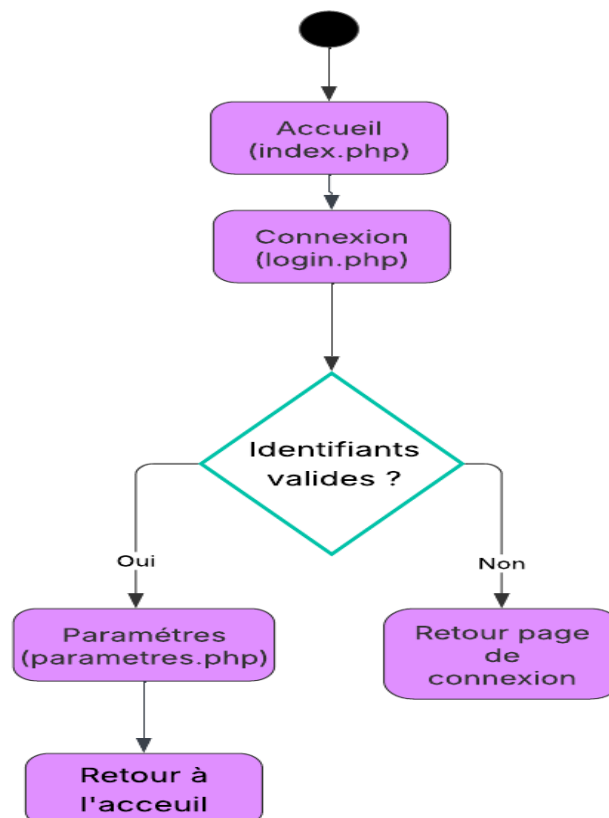
Se déconnecter

Chaque fonctionnalité a été isolée dans des fichiers PHP dédiés : (changer_mdp.php, modifier_seuils.php, export_csv.php).

5.5 Conception préliminaire

Objet	Fonction principale
ESP32	Fournir les mesures via HTTP (température, humidité...) en format JSON
Raspberry Pi	Interroger l'ESP32, stocker les données en base
Base de données MySQL	Enregistrer les mesures, seuils et identifiants
Interface web (PHP)	Afficher les données, modifier les seuils, exporter CSV

Ce diagramme ci-dessous montre la navigation logique de l'utilisateur au sein de l'IHM, depuis la page d'accueil jusqu'à la déconnexion.



Le modèle logique de données représente la structure interne de la base MySQL que j'ai utilisée dans ce projet. Il permet de décrire de manière claire les tables, les champs, leurs types, et leurs relations logiques.

Mesures	
id_mesures	INT (PK)
date_heure	DATETIME
temperature	FLOAT
hum_plante_1	FLOAT
hum_plante_2	FLOAT
hygrometrie	FLOAT

Seuils	
id_seuils	INT (PK)
temperature_min	FLOAT
temperature_max	FLOAT
humidite_min	FLOAT
humidite_max	FLOAT
hygrometrie_min	FLOAT
hygrometrie_max	FLOAT

Connexion	
id	INT (PK)
identifiant	VARCHAR
mdp	VARCHAR

La base est organisée autour de trois tables principales :

- Mesures : cette table enregistre chaque relevé transmis automatiquement par l'ESP32. Elle contient la température, l'humidité des deux plantes, l'hygrométrie ambiante, ainsi que la date et l'heure d'enregistrement. Chaque ligne correspond à un enregistrement complet effectué toutes les 5 minutes.

- Seuils : cette table contient les valeurs de seuils définies par l'utilisateur via l'interface. Ces seuils permettent d'indiquer des valeurs minimales ou maximales (par exemple température maximale autorisée ou humidité minimale à respecter).

Connexion : cette table contient les informations nécessaires à l'identification d'un utilisateur. Le mot de passe est stocké de manière sécurisée (hachée) afin de protéger l'accès à la page de paramétrage.

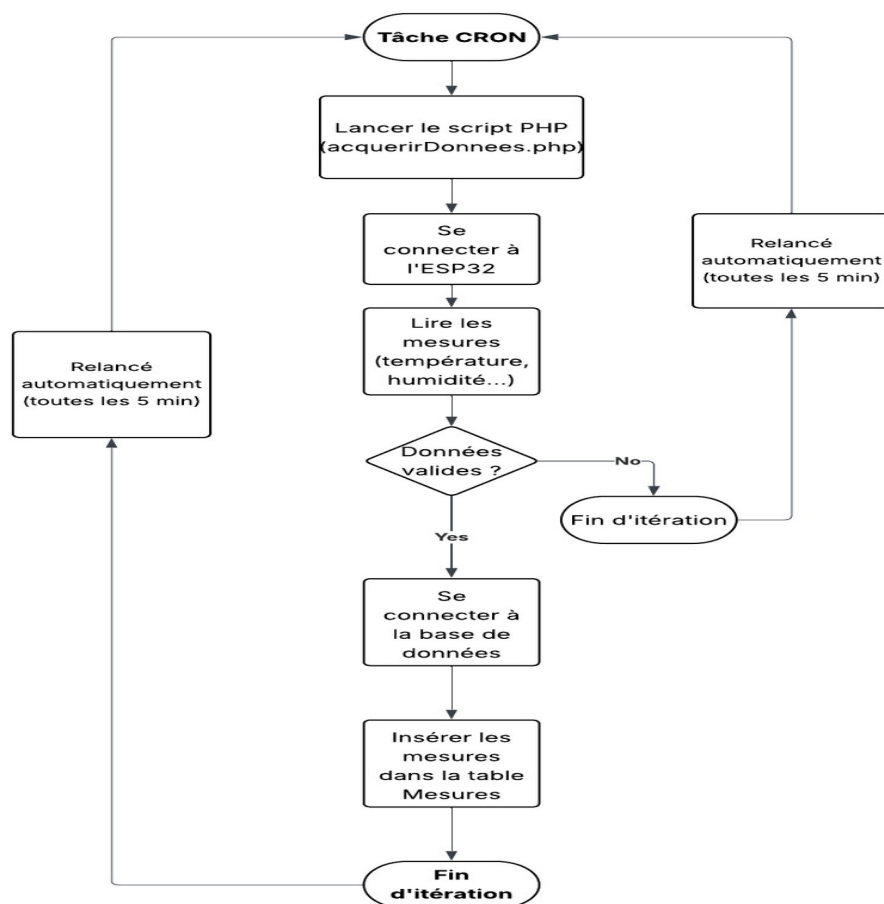
Ces tables sont indépendantes mais complémentaires. La table Mesures est la plus alimentée du système, car elle reçoit automatiquement de nouvelles lignes à intervalles réguliers. La table Seuils est modifiée manuellement par l'utilisateur via le formulaire de la page parametres.php. Quant à la table Connexion, elle n'est accessible que pour les opérations de connexion et de changement de mot de passe.

5.6 Conception détaillée

Le script principal s'appelle `acquerirDonnees.php` et fonctionne ainsi :

- Une tâche planifiée CRON l'exécute toutes les 5 minutes.
- Le script interroge l'ESP32 via `wget`.
- L'ESP32 renvoie une trame de type : `23.5;58.1;56.9;42.0`
- Le script découpe cette trame avec `explode()`.
- Les données sont insérées dans la base MySQL via une requête `INSERT`.

Ce diagramme montre le fonctionnement automatisé du script `acquerirDonnees.php` : une tâche CRON déclenche toutes les 5 minutes la récupération des données de l'ESP32, leur validation et leur enregistrement dans la base de données. Si la trame est invalide, l'itération s'arrête sans modification.



Fonction testée	Procédure	Résultat attendu
Connexion utilisateur	Login avec mauvais mot de passe	Message d'erreur + refus d'accès
Affichage graphique	Charger index.php après insertion	Données affichées sur le graphique
Export CSV	Clic sur bouton "Télécharger les mesures"	Fichier .csv généré et téléchargeable
Modification de seuils	Formulaire rempli et validé	Valeurs mises à jour en base

Sécurité de l'accès	Accès direct à parametres.php sans login	Redirection vers login.php
---------------------	--	----------------------------

5.7 Codage

L'ensemble des fichiers seront retrouvables sur un dépôt github a cette adresse :

https://github.com/RaphaelTep/Serre_Archive/tree/Raspberry%2BBDD

L'ensemble du code a été organisé de façon modulaire, dans des fichiers PHP et CSS distincts, pour garantir la lisibilité, la maintenance et la réutilisabilité du projet. Voici la structure et le rôle de chaque fichier :

Fichier	Rôle principal
acquerirDonnees.php	Script appelé automatiquement par CRON pour récupérer les données de l'ESP32
db_connect.php	Connexion centralisée à la base de données MySQL
index.php	Page d'accueil affichant les courbes des mesures via Chart.js
login.php	Page d'identification sécurisée avec formulaire
parametres.php	Page principale après connexion : modification des seuils, mot de passe, export CSV
changer_mdp.php	Traitement du formulaire de changement de mot de passe
modifier_seuils.php	Enregistrement des seuils dans la table Seuils
export_csv.php	Génération et téléchargement d'un fichier .csv des mesures
logout.php	Déconnexion de l'utilisateur et destruction de la session
hash_mdp.php	Script ponctuel pour générer le mot de passe haché au format sécurisé (hors production)

Les styles CSS sont séparées pour chaque page afin d'avoir une meilleure gestion visuelle :

CSS	Utilisation spécifique
style_index.css	Apparence de la page index.php (graphique + responsive)
style_login.css	Mise en forme du formulaire de connexion
style_parametres.css	Style complet de l'interface de gestion (seuils, CSV, mot de passe)

5.8 Résultats des tests unitaires

Composant	Test effectué	Résultat obtenu
ESP32 / requête HTTP	Réponse via wget	Trame correcte reçue
Script CRON	Attente de 5 minutes	Nouvelle ligne ajoutée automatiquement
Interface de connexion	Connexion / déconnexion	Sécurité fonctionnelle
Export CSV	Vérification du fichier généré	Données complètes et format correct
Sécurité des pages	Accès sans session	Redirection immédiate vers login.php

5.9 Bilan personnel

Tout au long du projet, j'ai adopté une méthode de travail progressive, structurée en plusieurs étapes. J'ai d'abord configuré le Raspberry Pi (installation de Raspbian, serveur Apache, base de données MySQL), puis j'ai mis en place le script d'acquisition automatisée via CRON. Une fois cette base fonctionnelle, j'ai développé les différentes interfaces web (accueil, connexion, paramétrage), tout en testant chaque étape. Les tâches ont été réparties dans le temps et documentées dans un carnet de bord. Cette organisation m'a permis d'avancer régulièrement et de valider chaque bloc de manière autonome.

J'ai également rencontré plusieurs difficultés techniques. Par exemple, les mises à jour du Raspberry étaient bloquées à cause d'une erreur liée aux clés APT, ce qui m'a obligé à intervenir manuellement sur les sources et les dépôts. J'ai aussi eu des erreurs d'exécution avec les scripts

CRON qui nécessitaient des chemins absolus et des droits d'accès particuliers. De plus, la gestion des sessions PHP et des redirections de sécurité (vers login.php) a nécessité plusieurs ajustements. Tous ces problèmes m'ont permis d'apprendre à déboguer, à rechercher des solutions efficaces et à sécuriser mon code.

Au final, cette expérience m'a permis de gagner en autonomie, en rigueur, et de renforcer mes compétences en développement web embarqué, en réseau local et en gestion de projet. J'ai également compris l'importance de bien structurer son code, de documenter son travail, et de tester en conditions réelles.

6 Intégration et tests

6.1 Intégration

Présenter le diagramme de déploiement final.

6.2 Tests

Présenter ici les tests d'intégration réalisés (condition, ...).

Ne peut pas être réalisé sans le matériel

Il s'agit de vérifier que les logiciels réalisés par 2 membres de l'équipe fonctionnent bien ensemble et peuvent être assemblés.

Action à tester	Tests à réaliser	Résultat(s) attendu(s)	Résultat(s) obtenu(s)	Remède(s)
Transmission des mesures de l'ESP32 vers le Raspberry Pi	Exécuter la tâche CRON automatiquement toutes les 5 minutes. Vérifier dans la base de données si une nouvelle ligne est insérée à chaque cycle.	Une trame contenant 4 mesures est reçue, découpée, et insérée dans la table Mesures. Un nouvel enregistrement apparaît toutes les 5 min.	Données insérées correctement à intervalle régulier. Présence confirmée dans phpmyadmin.	Aucun, fonctionnement conforme
Enregistrement et lecture des seuils entre le Raspberry Pi et l'ESP32	L'utilisateur modifie les seuils dans l'interface web. Vérifier que la base est mise à jour. Puis	Les nouvelles valeurs saisies dans le formulaire sont stockées dans la table Seuils. L'ESP32 lit et	Seuils bien modifiés dans la base de données. L'ESP32 les lit correctement via la requête	Aucun, fonctionnement conforme

	interroger l'ESP32 via le navigateur pour vérifier s'il reçoit les nouveaux seuils.	applique les seuils mis à jour.	http.	
--	---	---------------------------------	-------	--

7 Validation des attentes du point de vue client

A faire avec les plans de test de validation présenté au paragraphe 2 en remplissant les colonnes 4 et 5 du (des) tableau(x) des tests de validation.

Ne peut pas être réalisé sans le matériel

Il s'agit ici de réaliser les tests décrits au 2.3.

Action à tester	Tests à réaliser	Résultat(s) attendu(s)	Résultat(s) obtenu(s)

En tirer une conclusion par rapport aux objectifs fixés dans le dossier du sujet au niveau de la recette client à atteindre.

8 Bilan technique

8.1 Evolution(s) possible(s)

Pour l'ESP32 : ajouter un petit écran affichant les dernières données relevées par les différents capteurs

Pour le raspberry et la Base de Données, ajouter un système de journaling pour avoir des logs.

9 Dossier numérique

Voir https://github.com/RaphaelTep/Serre_Archive pour la documentation

Pour le code de l'ESP32 voir https://github.com/RaphaelTep/Serre_Archive/tree/ESP32

Voir https://github.com/RaphaelTep/Serre_Archive/tree/Raspberry%2BBDD pour le code du Raspberry